Evaluating Post-Quantum Key Exchange Algorithms in WireGuard

A Comparative Study of ML-KEM and HQC

Ludvig Järvi

supervised by
name lastname

Department of Computer Science, Electrical and Space Engineering
Luleå University of Technology
Luleå, Sweden

May 6, 2026

Acknowledgements

Abstract

Keywords: WireGuard, Post-Quantum, PQ, Post-Quantum Cryptography, PQC, ML-KEM, HQC,

Contents

Glossary

DH Diffie-Hellman.

HQC Hamming Quasi-Cyclic.

KEM Key-Encapsulation Mechanism.

ML-KEM Module-Lattice-Based Key-Encapsulation Mechanism.

NIST the National Institute of Standards and Technology.

PKE Public-Key Encryption.

PQ Post-Quantum.

PQC Post-Quantum Cryptography.

PSK Pre-Shared Key.

QKD Quantum Key Distribution.

VPN Virtual Private Network.

1 Introduction

In today's digital society, asymmetric encryption underpins nearly every aspect of global secure communications. Cryptographic protocols such as Transport Layer Security (TLS), Secure Shell (SSH), and Virtual Private Networks (VPNs) rely on asymmetric cryptography to securely establish shared secrets between previously untrusted parties. These cryptographic systems are based on the assumption that specific mathematical problems, notably integer factorization and discrete logarithms, are too hard for classical computers to solve efficiently.

However, this assumption may not hold in the long term. Rapid advances in quantum computing research suggest that large-scale, fault-tolerant quantum computers may become viable within the coming decades [1]. These are computers capable of leveraging the laws of quantum mechanics to perform quantum algorithms that achieves significant computational speedups for specific classes of problems in comparison to their classical counterparts. Such machines would be able to fundamentally break the asymmetric cryptography primitives used today.

Two quantum algorithms stand out in this cryptographic context. Peter Shor's algorithm can efficiently factor large integers and compute discrete logarithms [2], and Lov Grover's algorithm [3], which provides a quadratic speedup to brute-force key searches. The former directly breaks the public-key primitives underlying RSA, Diffie-Hellman and elliptic-curve cryptography, while the latter reduces the effective security level of symmetric ciphers and hash functions by half.

Once quantum computers reach this critical scale, they will likely be capable of decrypting nearly all historical and contemporary communications secured with current public-key systems. Moreover, adversaries can already capture encrypted traffic today with the intent of decrypting it later when quantum capabilities become available, a strategy commonly referred to as "harvest now, decrypt later." [4] This threat makes it essential to begin the transition toward quantum-resistant cryptography as soon as possible, since deploying new cryptographic standards across the Internet ecosystem will take many years.

To stay ahead of this threat, the cryptographic community is actively developing Post-Quantum Cryptography (PQC) algorithms. These are new types of "quantum-resistant" mechanisms designed to be secure against both today's computers and tomorrow's quantum machines. The algorithms are based on mathematical problems believed to remain hard for quantum computers, such as those derived from lattices, error-correcting codes, or hash functions. The National Institute of Standards and Technology (NIST) initiated, back in 2016, a global effort to identify and standardize these new methods [5], resulting in several promising candidates.

After several years of public evaluation, the NIST Post-Quantum Cryptography Standardization Process selected and standardized ML-KEM [6], a lattice-based Key-Encapsulation Mechanism (KEM), and is currently in the process of standardizing Hamming Quasi-Cyclic (HQC) [7], a code-based KEM. These schemes are considered primary building blocks for securing Internet communications in the quantum era.

While the theoretical security of these algorithms has been extensively studied, their practical deployment within real-world communication protocols remains an active research challenge. Post-quantum schemes often have larger key sizes, ciphertexts, and

computational overhead compared to classical cryptographic methods. Such differences can significantly affect performance-sensitive protocols that prioritize efficiency.

One such protocol is WireGuard [8], a modern VPN protocol designed for minimalism, simplicity, and high throughput. WireGuard currently relies on elliptic-curve cryptography, specifically Curve25519, as part of its authenticated key exchange mechanism. While this design offers excellent efficiency on classical hardware, elliptic-curve cryptography is vulnerable to quantum attacks due to its reliance on the discrete logarithm problem.

Integrating PQC into WireGuard therefore presents several unique challenges related to efficiency, compatibility, and code maintainability. These include managing increased message sizes, maintaining low handshake latency, and preserving the protocol's minimalist design philosophy. Existing solutions, such as Rosenpass and PQ-WireGuard prototypes, have demonstrated the feasibility of quantum-resistant VPN connections through external or hybrid key exchanges [9][10][11].

Yet, few studies have systematically compared the performance characteristics of NIST-approved algorithms when integrated into WireGuard's architecture.

This thesis addresses that gap by designing, implementing, and evaluating a hybrid WireGuard handshake that supports two NIST-selected Key-Encapsulation Mechanisms: ML-KEM and Hamming Quasi-Cyclic. The implementation enables an empirical comparison of their latency, computational overhead, memory usage, and message size, and contrasts the results against existing post-quantum WireGuard implementations. By quantifying these trade-offs, the work aims to provide practical insights into the real-world viability of post-quantum VPNs and contribute to the broader effort of securing Internet communications against future quantum threats.

1.1 Problem statement and research questions

How does NIST-approved Post-Quantum Key-Encapsulation Mechanisms, specifically ML-KEM and HQC, affect the performance, resource usage, and practical deployability of a hybrid WireGuard handshake compared to existing post-quantum approaches such as Rosenpass?

RQ1. How does NIST-selected post-quantum key encapsulation mechanisms, specifically ML-KEM and HQC, affect the performance of a WireGuard handshake?

RQ2. What impact do ML-KEM and HQC have on resource usage in WireGuard, particularly in terms of bandwidth, message size, and storage requirements?

RQ3. To what extent can ML-KEM and HQC be integrated into WireGuard without violating its core design principles, such as low latency, small handshake size, and simplicity?

RQ4. How does hybrid cryptographic approaches (combining classical Diffie-Hellman with PQC) compare to purely post-quantum approaches in terms of security, performance, and deployability?

RQ5. How does a hybrid WireGuard implementation using ML-KEM and HQC compare to existing post-quantum solutions, such as Rosenpass, in terms of performance and practical deployment?

RQ6. What trade-offs exist between security level and ciphertext size when selecting PQC algorithms for use in WireGuard?

1.2 Scope and limitations

Core scope:

- Userspace implementation in WireGuard-Go (no kernel changes)
- Evaluation on one primary hardware platform
- Comparative analysis and thesis documentation

Stretch goals:

- Cross-platform benchmarks (energy/performance scaling)
- Symbolic or formal modeling of the hybrid handshake
- Basic side-channel timing variance analysis

1.3 The scarlet thread

The central theme of this thesis is the integration of post-quantum cryptographic algorithms into existing secure communication protocols, with a specific focus on WireGuard.

The work is motivated by the emerging threat posed by quantum computing, which challenges the security of widely used public-key cryptographic schemes. In response, PQC introduces new algorithms that are believed to be resistant to quantum attacks, but these algorithms also introduce practical challenges related to performance, resource usage, and protocol compatibility.

The thesis follows a progression from theoretical foundations to practical application. First, the principles of classical and post-quantum asymmetric cryptography are introduced, along with the quantum threat that motivates the transition. Next, the NIST-selected post-quantum algorithms, ML-KEM and HQC, are described and analyzed in terms of their properties. Along with this, the overarching PQC challenges, as well as hybridization strategies, are discussed.

The main focus of the thesis is then the evaluation of how these algorithms can be integrated into the WireGuard protocol, particularly through hybrid cryptographic approaches. This includes analyzing the impact on performance, bandwidth usage, and protocol design constraints. Finally, the proposed approaches are compared to existing solutions, such as Rosenpass, in order to assess their practical viability.

The unifying thread throughout the thesis is the balance between security and practicality when transitioning to PQC in real-world systems.

1.4 Thesis structure

The remainder of this thesis is structured as follows.

Section 2.1 provides background on classical cryptography, including public-key cryptographic schemes such as RSA, Diffie-Hellman, and elliptic-curve cryptography. It also introduces key concepts such as key exchange and key encapsulation mechanisms.

Section 2.2 presents the quantum threat to modern cryptography. This includes a high-level overview of quantum computing and relevant algorithms such as Shor's and Grover's algorithms, as well as the "Harvest now, decrypt later." risk model.

Section 2.3 introduces Post-Quantum Cryptography. It introduces different families of post-quantum algorithms, with a focus on ML-KEM and HQC, and analyzes their properties, challenges, and trade-offs. Hybrid cryptographic approaches are also presented.

Section 3 focuses on the WireGuard protocol. It describes its core design principles, underlying cryptographic framework, handshake mechanism, and discusses the challenges of integrating PQC algorithms into this context.

Section 4 Related work

Section 5 Concurrent work

Section 6 Method

Section 7 presents an evaluation of hybrid post-quantum approaches in WireGuard. This includes analysis of performance and implementation considerations, as well as a comparison with existing solutions such as Rosenpass.

Finally, Section 8 and 9 concludes the thesis and discusses potential directions for future work.

2 Background

2.1 Asymmetric Cryptography

Asymmetric cryptography, also referred to as public-key cryptography, addresses the fundamental problem of symmetric cryptography in secure communications: how can two foreign parties establish a secure channel without first meeting up in person to exchange a secret key.

In order to solve this problem, asymmetric cryptography uses two mathematically related keys: a private key and a public key. The public key can be distributed freely, while the private key must remain secret. Messages encrypted with a public key can then only be decrypted using its corresponding private key, enabling secure communications without the need for a predetermined shared secret.

To illustrate this concept, imagine two people: Alice and Bob. Alice wants to send a secret message to Bob. To achieve this, they do the following:

1. Bob sends a box and an open padlock, to which only Bob has the key, to Alice.
2. Alice places her message in the box, locks it with the padlock, and sends it back to Bob.
3. Bob uses his key to unlock the box and is able to read Alice's message.

Even if some adversary were to intercept the box on its way back to Bob, they would be unable to open it and read Alice's message since they don't have Bob's key.

This simple analogy describes the core idea of asymmetric cryptography: separating the methods of encryption and decryption. Here the padlock represents the public key, and Bob's key the private key. He can send copies of his padlock to anyone who wants to send him a secret message, knowing only he can open them.

In practice, these systems use specific mathematical problems known as trapdoor functions [12]. These are problems that are easy to compute in one direction, but computationally infeasible to reverse without hidden information (the private key).

Asymmetric cryptography is not used directly to encrypt large amounts of data due to its computational overhead. Instead, it is commonly used during the initial handshake phase of a protocol to establish a shared symmetric key. Once this shared secret has been established, symmetric encryption algorithms can be used to protect the bulk of the communication efficiently.

2.1.1 RSA and Diffie-Hellman

The first generation of widely adopted asymmetric systems relied on two primary mathematical hurdles. The RSA (Rivest-Shamir-Adleman) algorithm, introduced in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman [13], built on the mathematical difficulty of factoring large composite integers. While multiplying two large prime numbers is computationally trivial, determining the original factors from their product is believed to be infeasible for sufficiently large numbers using classical algorithms. This asymmetry forms the basis of the RSA trapdoor function.

The other fundamental primitive introduced to public-key cryptography was the Diffie-Hellman (DH) key exchange, introduced in 1976 by Whitfield Diffie and Martin Hellman [14]. Rather than encrypting messages directly, Diffie-Hellman introduced the concept of the secure key exchange. Thus enabling two parties to securely establish a shared secret over an insecure channel. The security of this protocol relies on the hardness of the discrete logarithm problem, which involves recovering an exponent from a modular exponentiation operation.

These methods defined the first thirty years of digital security, but as computational power grew, the required key sizes for RSA and standard DH became unmanageable, leading to a need for more efficient mathematics.

2.1.2 The Shift to Elliptic-Curve Cryptography

In the early 2000s, many cryptograpic protocols began transitioning toward Elliptic-Curve Cryptography (ECC). Instead of operating over finite fields of integers, ECC relies on the algebraic structure of elliptic curves over finite fields. The advantage of the Elliptic Curve Discrete Logarithm Problem (ECDLP) is that it's significantly harder to solve per bit of key size than the two previous mathematical problems used in RSA and DH.

For example, a 256-bit elliptic curve key, such as Curve25519 used by WireGuard, provides a security level equivalent to a 3072-bit RSA or DH key [15]. This efficiency is one of the primary reasons modern protocols have adopted ECC. Smaller key sizes reduce bandwidth requirements and computational overhead, enabling faster handshakes and smaller packet headers, which are particularly important for performance-sensitive systems such as VPN protocols.

Despite their efficiency, elliptic-curve systems remain fundamentally vulnerable to quantum attacks due to their reliance on discrete logarithm problems. This vulnerability forms one of the key motivations for the development of post-quantum cryptographic algorithms.

Recognizing this risk, NIST has initiated a long-term transition toward quantum-resistant cryptographic algorithms. Current guidance recommends that organizations begin migrating away from classical public-key mechanisms in the coming years, with many traditional systems expected to be phased out after approximately 2030 as post-quantum standards are deployed [16]. This planned transition highlights the need to evaluate how existing protocols that rely heavily on elliptic-curve cryptography, such as WireGuard, can adapt to post-quantum cryptographic primitives.

2.1.3 Key Exchange vs. Key Encapsulation

It's important to distinguish between the two main methods for establishing shared cryptographic keys.

Traditional key exchange protocols, such as Diffie-Hellman and its elliptic-curve variant, employs an interactive process in which both parties contribute to the generation of a shared secret. Instead of sending the secret directly, each party exchanges intermediate values derived from their private key. Using these values, both participants can independently compute the same shared key.

In contrast, asymmetric post-quantum algorithms, including ML-KEM and HQC, are almost exclusively designed as Key Encapsulation Mechanisms (KEMs). Rather than

performing a mutual exchange of values, KEMs follow a sender-receiver model, where the sender generates a random symmetric key and encapsulates it using the recipient's public key. This process produces a ciphertext that is transmitted to the recipient. Using the corresponding private key, the recipient can decapsulate the ciphertext and recover the same shared secret key.

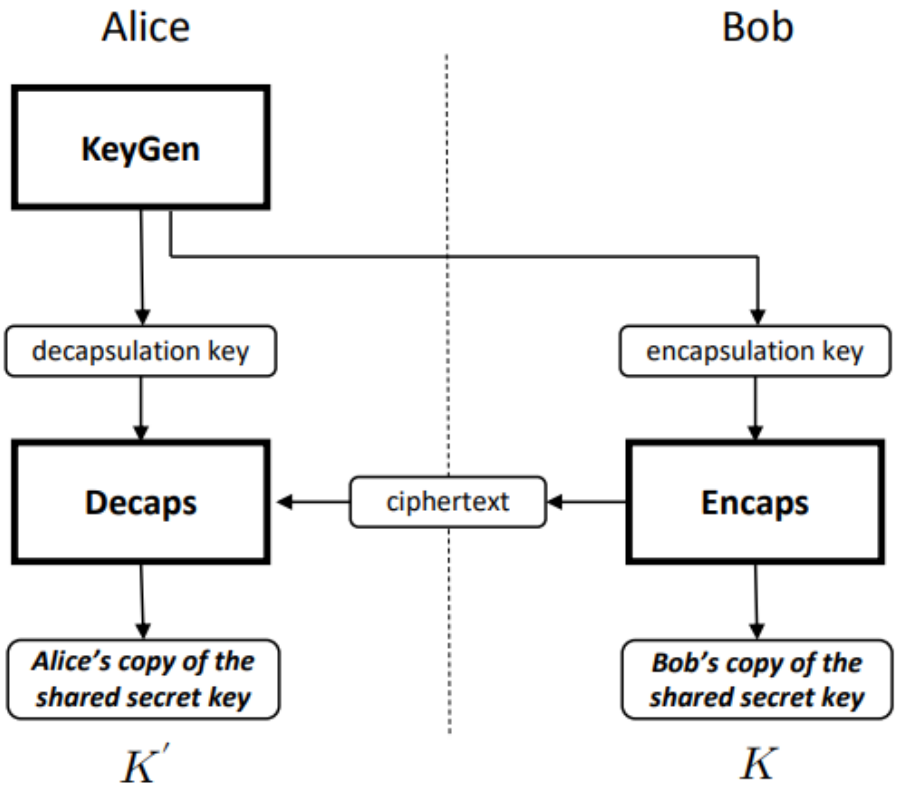

Figure 1: A simple view of key establishment using a KEM

While the end result is the same, the logistical flow is different. This distinction is a core challenge when integrating post-quantum KEMs into existing protocols like WireGuard, which was originally designed for the interactive key exchange flow of Diffie-Hellman.

Despite this difference in their structure, KEMs are generally preferred for post-quantum cryptography due to the nature of the mathematical problems on which post-quantum security is based. These do not support the bidirectional structure of a key exchange protocol. Instead, they lend themselves to a one-directional approach where one party can efficiently encapsulate a secret under a public key, while only the holder of the corresponding private key can recover it.

2.2 The Quantum Threat

As discussed in the introduction, the long-term security of modern cryptographic systems relies on the assumption that certain mathematical problems are computationally infeasible for classical computers to solve. However, the development of quantum computing challenges this assumption. Quantum computers operate according to the principles of quantum mechanics and are capable of executing algorithms that can outperform classical algorithms for specific types of problems.

There are two quantum algorithms that are especially relevant in this context: Shor's algorithm, which efficiently solves integer factorization and the discrete logarithm problem, and Grover's algorithm, which accelerates brute-force search. These algorithms are the frontrunners of the quantum threat to current cryptographic systems and are discussed in the following sections.

2.2.1 Quantum Computing

We must establish a high-level understanding of quantum computers work in order to understand how they are able to threaten current encryption.

A classical computer processes information using bits, binary values that exist in one of two states: 0 or 1. These bits are manipulated deterministically through electrical circuits made up of transistors and logic gates, which enables the computer to perform arithmetic and logical operations. At any given point in time, a classical system represents a single, well-defined state.

A quantum computer instead operates using quantum bits, commonly referred to as *qubits*, which has the ability to exist in a superposition of both states simultaneously. This means that a qubit represents a linear combination of the two states rather than a single deterministic value. When multiple qubits are combined, the system can represent a probability distribution of many possible states at once. For a system of n qubits, this corresponds to 2^n possible configurations, resulting in an exponentially large state space.

The other key property of quantum systems is entanglement, where the states of multiple qubits become correlated in a way such that the state of one qubit cannot be described independently of the others. Entanglement enables quantum computers to capture and exploit relationships between variables in ways that are not possible in classical systems.

Quantum algorithms leverage superposition, entanglement, and the manipulation of probability amplitudes to perform computations in a fundamentally different way than classical algorithms. By carefully controlling how probability amplitudes evolve during a computation, quantum algorithms can amplify the likelihood of correct solutions while cancelling out incorrect ones. This is what allows quantum computers to achieve significant computational speedups compared to their classical counterparts [17].

Quantum algorithms leverage superposition, entanglement, and the manipulation of probability amplitudes to perform certain classes of computations in a fundamentally different way than classical algorithms. By carefully controlling how probability amplitudes evolve during computation, these algorithms can amplify the likelihood of correct solutions while cancelling out incorrect ones. This interference structure is what enables quantum computers to achieve significant speedups over classical computation for problem classes where it can be meaningfully exploited [17].

2.2.2 Shor's Algorithm

One of the most significant breakthroughs in quantum computing for cryptography was the discovery of Shor's algorithm by Peter Shor in 1994 [2]. The algorithm provides an efficient quantum method for solving the two mathematical problems that are central to modern public-key cryptography: integer factorization and the discrete logarithm problem.

In order to fully grasp the impact of Shor's algorithm, let's first consider the best known classical algorithm for solving these problems. The General Number Field Sieve (GNFS) operates in sub-exponential but superpolynomial time [18], which means that every additional digit in an integer makes the factorization time disproportionately longer. While significantly faster than naive approaches, the computational cost still grows at an infeasible rate for larger key sizes. The largest RSA integer factored to date is RSA-250, an 829-bit number factored in February 2020 using CADO-NFS, an optimized software implementation of GNFS [19]. This required approximately 2700 CPU core-years [20], illustrating that classical factoring remains computationally prohibitive at the key sizes used in practice.

Shor's algorithm completely demolishes this notion of infeasability. By leveraging the principles of quantum mechanics, it is able to factor large integers and solve discrete logarithms in polynomial time. It accomplishes this by reducing the problem into a period-finding problem. Given an integer N, the algorithm selects a random value and constructs a function whose period is related to the factors of N. While finding such periods is computationally difficult for classical computers, quantum computers can solve this efficiently using the Quantum Fourier Transform (QFT).

The practical implications of Shor's algorithm have been extensively explored and optimized in later research. A widely cited paper by Craig Gidney and Martin Ekerå demonstrated in 2021 that factoring a 2048-bit RSA integer could be feasible using a quantum computer with around 20 million physical qubits and 8 hours of computation time, assuming realistic error rates, cycle times and reaction times [21]. This paper has recently recieved a follow-up by Gidney alone where he demonstrated that the same task could accomplished in less than a week using less than a million noisy qubits under the same assumed constraints [22].

The existence of Shor's algorithm therefore represents a fundamental threat to modern public-key cryptography. Since protocols such as TLS, SSH, and VPN systems rely on RSA, Diffie-Hellman, or elliptic-curve cryptography for secure key establishment, the availability of large-scale quantum computers would render these systems insecure.

2.2.3 Grover's Algorithm

Grover's algorithm [3] addresses a different class of problems than Shor's algorithm, specifically unstructured search. Given a search space of size N, a classical brute-force algorithm requires $\mathcal{O}(N)$ evaluations to find a target element. Grover's algorithm reduces this complexity to $\mathcal{O}(\sqrt{N})$ by leveraging quantum amplitude amplification.

In the context of cryptography, Grover's algorithm impacts symmetric-key primitives such as block ciphers and hash functions. But, unlike Shor's algorithm, Grover's algorithm does not completely break symmetric cryptographic schemes. Instead, it effectively halves their security level in terms of bit strength. This degradation can be mitigated by increasing key sizes. For example, a 128-bit symmetric key, which is considered secure against classical adversaries, would offer only 64 bits of security against a quantum adversary using Grover's algorithm. Consequently, doubling the key length (from 128 to 256 bits) is generally considered sufficient to maintain security in a post-quantum setting.

It is also important to note that practical implementations of Grover's algorithm face significant overhead due to error correction and circuit depth requirements, which further

limits its near-term applicability. But it remains a critical consideration when evaluating the long-term security of symmetric cryptographic systems.

2.2.4 "Harvest now, decrypt later."

A common misconception about the quantum threat is that it only becomes relevant when sufficiently large quantum computers become available, but this misconception overlooks an already active risk model known as the "harvest now, decrypt later" (HNDL) [4].

In this model, an adversary intercepts and stores encrypted communication today with the intention of decrypting them in the future once sufficiently powerful quantum computers exist. This strategy is particularly concerning for data with long-term confidentiality requirements, such as government communications, intellectual property, personal data, and critical infrastructure information.

It is important to understand what part of modern encrypted communication is vulnerable in this context. While data at rest is protected by symmetric encryption algorithms such as AES, which are considered resistant to quantum attacks, the asymmetric public-key algorithms used during the key exchange are the primary target. If an adversary can intercept the key exchange step of a session protected by TLS for example, they can store it and later use a quantum computer to recover the symmetric session key and decrypt all the session data. The implication is that even data which appears to be well-protected at rest may be vulnerable if the key exchange that established its protection was recorded.

(Example of HNDL: Recent BGP hijackings)

The HNDL threat has driven significant regulatory action across multiple jurisdictions, each establishing strict timelines for the transition to post-quantum cryptography directly motivated by HNDL risk.

In the United States, the NSA's Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) establishes a phased migration schedule for National Security Systems. Traditional networking equipment such as VPNs and routers is required to support and prefer CNSA 2.0 algorithms by 2026, and to use them exclusively by 2030. Large-scale public-key infrastructure systems, such as TLS, follow a slightly later schedule, with mandatory preference by 2030 and exclusive use by 2033 [23].

In Europe, the European Commission published a coordinated PQC implementation roadmap, which establishes a timeline for the transition to PQC for all member states. Under this roadmap, all member states are expected to have begun transitioning to post-quantum cryptography by the end of 2026, with critical infrastructure systems fully transitioned to PQC no later than the end of 2030 [24].

At the standards level, NIST's deprecation timelines add further urgency. Classical quantum-vulnerable algorithms providing less than 128 bits of security are set to be deprecated after 2030 and disallowed after 2035, with algorithms above the 128-bit level set to follow the same eventual prohibition [25].

In summary, the "harvest now, decrypt later" threat establishes that the urgency of post-quantum migration is determined not only by the arrival of quantum computers, but also by the confidentiality requirements of data being transmitted and stored today. The combination of already-active data collection, long cryptographic migration timelines, and

firm regulatory deadlines means that organizations handling sensitive data must treat the transition to quantum-resistant cryptography as an active and ongoing priority.

2.3 Post-Quantum Cryptography

Post-Quantum Cryptography (PQC) refers to cryptographic algorithms that are designed to remain secure against attacks from both classical and quantum computers. The goal is to replace or complement current cryptographic primitives such as RSA, Diffie-Hellman, and ECC. In many cases, PQC schemes are designed to be integrated into existing protocols with minimal changes, although this still requires adaptations due to fundamental differences in protocol logic and accommodation for larger key sizes.

PQC algorithms are based on mathematical problems that are believed to be hard even for quantum computers. The main categories include lattice-based, code-based, multivariate, and hash-based cryptography. This thesis focuses on the two NIST-selected approaches: lattice-based cryptography (ML-KEM) and code-based cryptography (HQC).

2.3.1 Lattice-Based Cryptography (ML-KEM)

Formerly known as CRYSTALS-Kyber [26], now standardized under the name ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism) in the Federal Information Processing Standards (FIPS) publication 203 [6], is a lattice-based KEM finalized by NIST in August 2024.

ML-KEM is based on the hardness of lattice problems, specifically the so-called Module Learning With Errors (MLWE) problem. The core idea is that a system of linear equations with a small amount of added random noise is computationally hard to solve, even for a quantum computer.

One of the main advantages of ML-KEM is its strong performance and relatively moderate key sizes when compared to other PQC candidates. For example, ML-KEM-768 has a public key size of around 1184 bytes, which is manageable in protocols like TLS. Key generation and encapsulation are also significantly faster than RSA. These properties make ML-KEM suitable for practical deployment, which is why NIST selected it as the primary standard for post-quantum key establishment.

2.3.2 Code-Based Cryptography (Hamming Quasi-Cyclic)

Hamming Quasi-Cyclic (HQC) [7] is a code-based KEM selected by NIST in March 2025 as a backup standard to ML-KEM [27]. A draft standard is expected in 2026, with finalization planned for 2027.

The security of HQC is based on the hardness of decoding random quasi-cyclic codes, specifically the Quasi-Cyclic Syndrome Decoding (QCSD) problem. This problem is a structured variant of the general Syndrome Decoding (SD) problem, which is known to be NP-hard. Unlike older code-based schemes such as McEliece [28], HQC does not require hiding the structure of the code. Instead, it uses publicly known quasi-cyclic codes, and its security relies entirely on the hardness of the decoding problem itself.

HQC was selected primarily to provide *cryptographic diversity*. Since ML-KEM is based on lattice problems, a future cryptanalytic breakthrough targeting lattice-based cryptography

could potentially affect all systems relying on ML-KEM. By standardizing HQC, which is based on a completely different mathematical foundation, NIST ensures that a backup remains available if ML-KEM is found to be vulnerable. NIST cited HQC's solid and well-analyzed construction as reasons it was preferred over other fourth-round candidates.

2.3.3 Practical Challenges

Despite their security advantages, PQC algorithms introduce several practical challenges that must be addressed before they can be widely deployed.

The main challenge is the significantly larger message sizes compared to classical cryptography. PQC schemes typically produce larger public keys, ciphertexts, and signatures, which increases the size of protocol messages. This can create compatibility issues with existing systems, especially older implementations that impose strict limits on message sizes. These larger key materials may require adjustments to how handshake messages are structured and transmitted.

Another challenge is the integration of PQC algorithms into existing protocols as these were originally designed for classical cryptographic primitives and do not always directly support the properties of PQC schemes. As a result, careful modifications are required to ensure correct functionality and maintain security guarantees. In the case of TLS 1.3, for instance, changes are needed to accommodate larger key shares and to support hybrid key exchange mechanisms.

Finally, PQC implementations must be designed with strong resistance to side-channel attacks. Like classical cryptographic algorithms, PQC schemes can leak sensitive information through timing differences, power consumption, or electromagnetic emissions. Recent research has demonstrated practical side-channel attacks against implementations of HQC, which highlights the importance of constant-time implementations and careful engineering practices.

2.3.4 Hybridization Strategies

Because PQC algorithms are relatively new and have not yet undergone the same level of long-term cryptanalysis as classical schemes, a common deployment strategy is to combine a classical key exchange with a PQC KEM in a single handshake. This approach is known as a *hybrid key exchange*.

In a hybrid scheme, both a classical algorithm (such as elliptic-curve Diffie-Hellman) and a PQC KEM (such as ML-KEM) are executed in parallel. The resulting shared secrets are then combined using a KDF to derive the final session key. The key property of this approach is that security holds as long as *at least one* of the two components remains secure. If the PQC algorithm contains an unknown vulnerability, the classical component continues to provide security against current adversaries. Conversely, if a sufficiently powerful quantum computer becomes available in the future, the PQC component ensures that the session key remains protected.

Hybrid key exchange is currently the industry-standard approach for deploying PQC in practice. It is already used in TLS 1.3, where hybrid key exchange mechanisms such as `X25519MLKEM768` (combining X25519 with ML-KEM-768) have been standardized by the IETF [29], and is being deployed by providers such as Cloudflare [30].

3 WireGuard

WireGuard is a modern, free and open-source VPN protocol developed by Jason A. Donenfeld [8], and first presented at the Network and Distributed System Security Symposium (NDSS) in 2017. It was conceived as a reaction to the complexity and bloat of existing VPN solutions such as IPsec and OpenVPN, that makes them notoriously difficult to configure correctly and whose large codebases make security auditing require significant effort. WireGuard's philosophy is therefore built around three core principles: simplicity, high performance, and strong cryptography.

The primary C implementation of WireGuard is written in less than 4000 lines of code, excluding cryptographic primitives, compared to the 100 000 to 600 000 lines of the OpenVPN and IPsec implementations. This makes the codebase small enough for a single person to read and understand in full, which has multiple direct benefits: a smaller attack surface, easier auditing, and fewer places for subtle bugs to hide.

Unlike IPsec or OpenVPN, WireGuard does not offer any choice of cryptographic primitives. The suite is fixed. This eliminates the risk of misconfiguration and the complexity of negotiating cipher suites. The protocol uses Curve25519 for key exchange, ChaCha20-Poly1305 for authenticated symmetric encryption, BLAKE2s for hashing, and SipHash24 for hashtable keys.

WireGuard operates over the User Datagram Protocol (UDP) and uses a lightweight handshake mechanism to establish secure connections. A key design goal is to achieve fast connection establishment with minimal latency, typically requiring only one round-trip time (1-RTT).

Due to its efficiency and simplicity, WireGuard has been integrated into the Linux kernel, is used as the underlying protocol in commercial VPN services such as NordVPN (NordLynx) and Mullvad, and is also widely deployed across mobile devices and cloud infrastructure.

3.1 The Noise Protocol Framework

WireGuard is built on the Noise Protocol Framework [31], a public-domain framework for constructing secure communication protocols based on Diffie-Hellman key exchange. The framework defines a series of handshake patterns that describe how two parties initiate communication, exchange keys, and establish shared secrets.

WireGuard specifically uses the NoiseIK pattern. In this pattern, the initiator already knows the static public key of the responder, which allows for authenticated key exchange. This design provides mutual authentication while keeping the handshake efficient.

3.2 Handshake and Cryptographic Primitives

Optional pre-shared key...

The handshake uses the following set of modern cryptographic primitives:

- **Curve25519** for Diffie-Hellman key exchange. Curve25519 is an elliptic-curve DH function designed for both high performance and resistance to implementation errors. A static Curve25519 key is only 32 bytes, making it compact enough to fit easily in protocol messages.

- **ChaCha20-Poly1305** for authenticated symmetric encryption. ChaCha20 is a stream cipher and Poly1305 is a message authentication code; combined, they provide both confidentiality and integrity. This combination is particularly fast on hardware without AES acceleration, such as mobile processors.
- **BLAKE2s** for hashing. BLAKE2s is a fast cryptographic hash function used for key derivation and as part of the handshake's transcript hashing.
- **HKDF** for key derivation, built on top of BLAKE2s, used to derive the final session keys from the accumulated DH outputs.

3.3 Go Implementation

While the main WireGuard implementation is a Linux kernel module written in C, a userspace implementation exists for platforms where kernel-level access is unavailable or undesirable. This is `wireguard-go`, the official userspace implementation written in Go, which exposes the same configuration interface as the kernel module and is used on macOS, Windows, iOS, and Android.

The main motivation for a userspace implementation in Go is portability and ease of integration. Go provides cross-platform support, a rich standard library, and easier interoperability with higher-level application code compared to kernel C code. This makes it straightforward to embed WireGuard as a library in a larger application, as is common in VPN clients.

However, userspace implementations come with a performance trade-off. Because each packet must cross the boundary between user space and the kernel's network stack through a TUN interface, there is additional overhead compared to the kernel module, where packet processing happens entirely in kernel space. For this reason, `wireguard-go` is recommended primarily for platforms where the kernel module is not available, and the kernel module remains the preferred choice on Linux servers.

A notable alternative to `wireguard-go` is BoringTun, a userspace implementation written in Rust that is developed by Cloudflare [32]. BoringTun is deployed on millions of iOS and Android consumer devices as well as thousands of Cloudflare Linux servers. Because Rust avoids garbage collection and provides fine-grained control over memory, it can achieve better and more predictable performance than the Go implementation for packet-heavy workloads.

In the context of this thesis, the Go implementation is of particular interest because it allows PQC cryptographic libraries (which are often written in Go or have Go bindings) to be integrated directly into the WireGuard codebase without requiring kernel modifications. This makes experimental PQC integration significantly easier compared to modifying the C kernel module.

3.4 PQC Implementation Challenges

Integrating PQC into WireGuard introduces several challenges that stem from a fundamental architectural mismatch. WireGuard's handshake is built around the Diffie-Hellman key exchange, where both parties contribute a key share and independently derive the same shared secret. But since PQC schemes are typically structured as KEMs, where one party

generates a shared secret and encrypts it for the other, this means that PQC algorithms cannot simply replace the DH steps in the existing handshake without overarching changes to its code.

A more immediate practical challenge is the size of PQC key material. A Curve25519 public key is 32 bytes, while an ML-KEM-768 public key is much larger at 1,184 bytes and its ciphertext at 1,088 bytes. This matters because WireGuard is specifically designed to keep its handshake messages small enough to fit within a single unfragmented UDP packet. For IPv6, the minimum guaranteed MTU along any path is 1,280 bytes, leaving only 1,232 bytes for the payload after subtracting the IP and UDP headers. Fitting a full PQC handshake, especially a hybrid one that also carries classical key material, within this budget is very difficult, and exceeding it forces fragmentation, which causes the protocol to no longer maintain its 1-RTT handshake core design property.

4 Related Work

The foundational work on post-quantum WireGuard was presented by Hülsing, Ning, Schwabe, Weber, and Zimmermann at IEEE S&P 2021 [10]. Unlike most earlier work on post-quantum security for real-world protocols, their variant does not only address post-quantum confidentiality, but also targets full post-quantum authentication. To achieve this, they replace the Diffie-Hellman-based handshake with a more generic construction using only KEMs, resulting in a protocol that provides post-quantum security for both key exchange and identity authentication.

The core design challenge they address is the MTU constraint described in Section 3.4: because WireGuard operates over UDP and assumes that handshake messages fit within a single unfragmented IPv6 packet, the key material contributed by PQC algorithms must fit within 1232 bytes. To satisfy this constraint, they instantiate their construction using Classic McEliece as the CCA-secure KEM and a passively secure variant of Saber, carefully selected so that the combined key material fits within the available space. A significant practical limitation of this choice is the large public key size of Classic McEliece, which substantially increases the server's memory footprint and complicates deployment in resource-constrained environments such as the Linux kernel.

PQ-WireGuard serves as an important performance reference point as the algorithms it uses, Classic McEliece and Saber, were the best available candidates at the time, but have since been superseded by the NIST-standardized ML-KEM and the newly selected HQC. This thesis revisits the performance question using these standardized algorithms, allowing for a more relevant comparison against the current state of post-quantum standardization.

Rosenpass [9], developed by Varner, Lipp, Schmidt, Peischl, and Zaeske, takes a different approach to the integration problem. Rather than replacing the WireGuard handshake entirely, Rosenpass runs as a separate process alongside WireGuard and performs a post-quantum key exchange independently. The resulting shared secret is then supplied to WireGuard as its pre-shared symmetric key (PSK), producing a hybrid connection where security holds as long as either the classical WireGuard handshake or the Rosenpass key exchange remains secure.

This sidecar architecture sidesteps the MTU problem entirely, since Rosenpass is not constrained by WireGuard's packet size budget. It also avoids any modification to WireGuard's codebase, making it practical to deploy today without kernel changes. The protocol builds on the PQ-WireGuard design but improves it by adding resistance against state disruption attacks, where an adversary replays the first handshake message to disrupt the responder's state. Rosenpass addresses this by returning an encrypted cookie to the initiator, so that the responder does not commit state until the initiator proves reachability. The implementation is written in Rust and uses Classic McEliece for long-term keys and ML-KEM for ephemeral keys.

For this thesis, Rosenpass is relevant as a deployed reference implementation that already uses ML-KEM in a production context. Its PSK-based integration strategy also represents a natural performance baseline: since it adds a separate handshake on top of WireGuard rather than replacing it, the combined latency is higher than a fully integrated solution. The performance results in this thesis can therefore be interpreted in the context of whether a deeper integration actually yields a meaningful latency improvement over the Rosenpass approach.

The most recent related work is by Hashimoto, Katsumata, Niot, and Wiggers, to appear at IEEE S&P 2026 [11]. This paper revisits the original PQ-WireGuard design by Hülsing et al. and improves it across three dimensions: protocol design, computational security, and efficiency.

On the design side, the paper introduces the novel idea of *reinforced KEMs* (RKEMs), which provide a principled way to replace DH operations within the Noise framework with KEM operations. The original PQ-WireGuard handled this substitution in an ad-hoc manner, and the RKEM abstraction formalizes and corrects the way KEM outputs are absorbed into the handshake transcript and key derivation chain.

On the security side, the paper extends the computational security analysis beyond what was covered in the original work, which relied on the symbolic model for several of its security properties.

On the efficiency side, the RKEM-based redesign eliminates the need for Classic McEliece entirely. By using two instances of ML-KEM, the large public key sizes and associated memory overhead of the original design are avoided, making the protocol significantly more practical for deployment.

This paper represents the current state of the art for formally analyzed post-quantum WireGuard designs, and it uses ML-KEM as its primary building block, the same algorithm that is evaluated in this thesis. While this thesis does not implement the RKEM construction, the performance characteristics of ML-KEM measured here are directly relevant to understanding the practical cost of the approach proposed by Hashimoto et al.

5 Concurrent Work

Tailscale

Other VPNs implementing PQC

NIST Post-Quantum Cryptography Standardization Process [5] and HQC standardization process

Regev's Algorithm [33] and discrete logarithms extension [34]

6 Method

WireGuard-Go + liboqs

Use Level V parameters for all classification levels (NSA Commercial National Security Algorithm Suite 2.0).

6.1 Benchmarking setups

- WireGuard native macOS client
- WireGuard-Go default
- WireGuard-Go PQC
  - ML-KEM
  - HQC
- WireGuard-Go Hybrid
- WireGuard-Go Linux VM

6.2 Program

7 Results

8 Discussion

As this field of study is constantly evolving

8.1 Active Research

Bring up current research

- HQC Standardization
- New KEMs and public key exchange methods

9 Conclusion

References

[1] M. Swayne. "Quantum Computing Roadmaps: A Look at the Maps and Predictions of Major Quantum Players." Accessed: 2026-03-10, The Quantum Insider. [Online]. Available: https://thequantuminsider.com/2025/05/16/quantum-computing-roadmaps-a-look-at-the-maps-and-predictions-of-major-quantum-players/

[2] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Review*, vol. 41, no. 2, pp. 303–332, 1999. DOI: 10.1137/S0036144598347011 eprint: https://doi.org/10.1137/S0036144598347011. [Online]. Available: https://doi.org/10.1137/S0036144598347011

[3] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, ser. STOC '96, Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, pp. 212–219, ISBN: 0897917855. DOI: 10.1145/237814.237866 [Online]. Available: https://doi.org/10.1145/237814.237866

[4] J. Mascelli and M. Rodden, ""Harvest Now Decrypt Later": Examining Post-Quantum Cryptography and the Data Privacy Risks for Distributed Ledger Networks," Board of Governors of the Federal Reserve System, Washington, DC, Tech. Rep. 2025-093, 2025. DOI: 10.17016/FEDS.2025.093 [Online]. Available: https://doi.org/10.17016/FEDS.2025.093

[5] National Institute of Standards and Technology, *Post-Quantum Cryptography (PQC) Standardization Process*, https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization, [Online; accessed 19-February-2026], 2025.

[6] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," in *NIST, Tech. Rep.*, 2024. DOI: 10.6028/NIST.FIPS.203 [Online]. Available: https://doi.org/10.6028/NIST.FIPS.203

[7] C. Aguilar-Melchor et al., "Hamming Quasi-Cyclic (HQC) Specifications," HQC Team, Tech. Rep., 2025, Version dated 2025-08-22. [Online]. Available: https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf

[8] J. A. Donenfeld, "Wireguard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, The Internet Society, 2017. DOI: 10.14722/ndss.2017.23160 [Online]. Available: https://www.wireguard.com/papers/wireguard.pdf

[9] K. Varner, B. Lipp, L. Schmidt, M. Peischl, and W. Zaeske, "Rosenpass," 2024. [Online]. Available: https://rosenpass.eu/whitepaper.pdf

[10] A. Hülsing, K.-C. Ning, P. Schwabe, F. J. Weber, and P. R. Zimmermann, "Post-quantum wireguard," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 304–321. DOI: 10.1109/SP40001.2021.00030

[11] K. Hashimoto, S. Katsumata, G. Niot, and T. Wiggers, "Revisiting PQ WireGuard: A comprehensive security analysis with a new design using Reinforced KEMs," in *IEEE Security & Privacy 2026*, 2026. [Online]. Available: https://thomwiggers.nl/publications/pq-wireguard/

[12] A. C. Yao, "Theory and application of trapdoor functions," in *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, 1982, pp. 80–91. DOI: 10.1109/SFCS.1982.45

[13] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978. DOI: 10.1145/359340.359342

[14] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976. DOI: 10.1109/TIT.1976.1055638

[15] E. Barker, "Recommendation for Key Management: Part 1 – General," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. 800-57 Part 1 Rev. 5, May 2020. DOI: 10.6028/NIST.SP.800-57pt1r5 [Online]. Available: https://doi.org/10.6028/NIST.SP.800-57pt1r5

[16] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, "Transition to Post-Quantum Cryptography Standards," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. NIST IR 8547 (Initial Public Draft), Nov. 2024. DOI: 10.6028/NIST.IR.8547.ipd [Online]. Available: https://doi.org/10.6028/NIST.IR.8547.ipd

[17] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, "Quantum Amplitude Amplification and Estimation," *arXiv preprint arXiv:quant-ph/0005055*, May 2000. [Online]. Available: https://arxiv.org/abs/quant-ph/0005055

[18] J. P. Buhler, H. W. J. Lenstra, and C. Pomerance, "Factoring integers with the number field sieve," in *The Development of the Number Field Sieve*, A. K. Lenstra and H. W. J. Lenstra, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 50–94, ISBN: 978-3-540-47892-8.

[19] The CADO-NFS Development Team, *CADO-NFS, An Implementation of the Number Field Sieve Algorithm*, https://cado-nfs.gitlabpages.inria.fr, Development version, 2024.

[20] F. Boudot, P. Gaudry, A. Guillevic, N. Heninger, E. Thomé, and P. Zimmermann, "Comparing the Difficulty of Factorization and Discrete Logarithm: A 240-Digit Experiment," in *Advances in Cryptology – CRYPTO 2020*, ser. Lecture Notes in Computer Science, vol. 12171, Cham: Springer, 2020, pp. 62–91. DOI: 10.1007/978-3-030-56784-2_3

[21] C. Gidney and M. Ekerå, "How to factor 2048-bit RSA integers in 8 hours using 20 million noisy qubits," *arXiv preprint arXiv:1905.09749*, Apr. 2021, Version 3. DOI: 10.22331/q-2021-04-15-433 [Online]. Available: https://arxiv.org/abs/1905.09749

[22] C. Gidney, "How to factor 2048-bit RSA integers with less than a million noisy qubits," *arXiv preprint arXiv:2505.15917*, May 2025. DOI: 10.48550/arXiv.2505.15917 [Online]. Available: https://arxiv.org/abs/2505.15917

[23] National Security Agency, "Announcing the Commercial National Security Algorithm Suite 2.0," National Security Agency, Tech. Rep. PP-22-1338, Sep. 2022, Cybersecurity Advisory, Version 1.0. [Online]. Available: https://media.defense.gov/2025/May/30/2003728741/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS.PDF

[24] EU PQC Workstream, "A coordinated implementation roadmap for the transition to post-quantum cryptography," European Union, Tech. Rep. Part 1, Version 1.1, Jun. 2025, Published June 11, 2025. [Online]. Available: https://digital-strategy.ec.europa.eu/en/library/coordinated-implementation-roadmap-transition-post-quantum-cryptography

[25] D. Moody, *NIST PQC: The Road Ahead*, Presentation, Cryptographic Technology Group, Accessed: 2026-03-24, National Institute of Standards and Technology, Mar. 2025. [Online]. Available: https://csrc.nist.gov/csrc/media/Presentations/2025/nist-pqc-the-road-ahead/images-media/rwcpqc-march2025-moody.pdf

[26] J. Bos et al., "CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM," in *Proceedings of the 2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018, pp. 353–367. DOI: 10.1109/EuroSP.2018.00032

[27] D. Moody et al., "Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. NIST IR 8545, Mar. 2025. DOI: 10.6028/NIST.IR.8545 [Online]. Available: https://doi.org/10.6028/NIST.IR.8545

[28] R. J. McEliece, "A Public-Key Cryptosystem Based on Algebraic Coding Theory," Jet Propulsion Laboratory, Pasadena, CA, Tech. Rep. 42-44, Feb. 1978.

[29] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, "Post-quantum hybrid ECDHE-MLKEM Key Agreement for TLSv1.3," Internet Engineering Task Force, Internet-Draft draft-ietf-tls-ecdhe-mlkem-04, Feb. 2026, Work in Progress, 12 pp. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-tls-ecdhe-mlkem/04/

[30] B. Westerbaan. "State of the Post-Quantum Internet in 2025." Accessed: 2026-03-24, Cloudflare. [Online]. Available: https://blog.cloudflare.com/pq-2025/

[31] Trevor Perrin, *The Noise Protocol Framework*, https://noiseprotocol.org/noise.html, [Online; accessed 5-March-2026], 2018.

[32] Cloudflare, Inc., *BoringTun: Userspace WireGuard Implementation in Rust*, https://github.com/cloudflare/boringtun, GitHub repository, accessed 2026-03-25, 2026.

[33] O. Regev, "An Efficient Quantum Factoring Algorithm," *arXiv preprint arXiv:2308.06572*, Aug. 2023. DOI: 10.48550/arXiv.2308.06572 [Online]. Available: https://arxiv.org/abs/2308.06572

[34] M. Ekerå and J. Gärtner, "Extending Regev's Factoring Algorithm to Compute Discrete Logarithms," *arXiv preprint*, Jan. 2024. [Online]. Available: https://arxiv.org/abs/2311.05545